

Ubuntu Server Console Power Management

Server Details

- **Hostname:** `imac-server`
- **Operating System:** Ubuntu 24.04.2 LTS (Noble Numbat)
- **Role:** Headless server (no graphical desktop environment) with a directly connected monitor for console access.

Objective

To configure the `imac-server`'s physical monitor to automatically power off after 5 minutes of inactivity, and to re-activate upon keyboard input, while ensuring the server remains fully operational and responsive (e.g., via SSH).

Problem Statement

Initially, the `consoleblank=300` kernel parameter was set in `/etc/default/grub`, intended to blank the text console after 300 seconds (5 minutes). However, the physical monitor remained on, displaying a blank screen but not truly entering a power-saving state, consuming unnecessary power. Manual attempts to control the display's power state were met with various errors depending on the execution context.

Root Cause Analysis

The core issue stemmed from the distinction between kernel console blanking and true display power management (DPMS) at the hardware level, as well as the execution context of the commands.

1. **`consoleblank` Limitation:** While `consoleblank=300` correctly tells the kernel to stop drawing to the console after inactivity (effectively "blanking" the screen), it does not always send a DPMS signal to the monitor to power it down. The monitor might still receive a blank video signal, keeping its backlight on.
2. **`setterm` Tooling:** The `setterm` utility is designed for console-specific operations, including DPMS control.
 - **"Command not found":** Initially, `setterm` was not installed on the minimal server setup, requiring installation of the `kbd` package.

- **setterm: argument error: auto:** The `-powersave` option for `setterm` expects specific arguments (on, off, vsync, hsync), not auto. This was corrected to `-powersave on`.
 - **Execution Context (SSH vs. Physical Console):** `setterm` requires direct access to a physical console (TTY) device. When run over an SSH session (which uses a pseudo-terminal like `xterm-256color`), it reports errors like "terminal xterm-256color does not support --blank" or "Inappropriate ioctl for device" because it cannot interact with the actual hardware display.
 - **"Inappropriate ioctl for device" (Persistent):** Even when run from the physical console, `setterm` might report this error. This indicates that the specific low-level system call (`ioctl`) `setterm` uses to command DPMS might not be directly supported or exposed by the graphics driver in text mode. However, in this specific server's case, despite the error, the screen *did* power off, implying the command still triggered the desired behavior through other underlying mechanisms or side effects. This error was ultimately deemed ignorable as long as the functionality was achieved.
3. **systemd Service Context:** When attempting to automate `setterm` at boot using a `systemd` service:
- **setterm: \$TERM is not defined:** Services run by `systemd` typically have a minimal environment and are not associated with a specific terminal, hence the `TERM` variable (which defines the terminal type, e.g., `linux` for a console) is not set. `setterm` requires `TERM` to be defined to function correctly.

Solution Implemented

The solution involved creating a `systemd` service to execute the `setterm` command at boot, specifically directing it to operate on the physical console and providing the necessary environment variables.

1. Console Power Management Command

The command used to control the display's power state is:

```
/usr/bin/setterm -blank 5 -powerdown 5 -powersave on
```

- `-blank 5`: Sets the console blanking time to 5 minutes (300 seconds). This clears the text from the screen.
- `-powerdown 5`: Attempts to send a DPMS power-down signal to the monitor after 5 minutes of inactivity, instructing the monitor to enter its lowest power state (typically turning off the backlight).
- `-powersave on`: Explicitly enables DPMS features for the console.

2. systemd Service for Persistence

A custom systemd service unit was created to ensure this command runs reliably at every boot.

- **Service File Path:** `/etc/systemd/system/console-power-save.service`

- **Service File Content:**

```
[Unit]
Description=Set Console Power Saving
After=multi-user.target

[Service]
Type=oneshot
ExecStart=/usr/bin/setterm -blank 5 -powerdown 5 -powersave on < /dev/tty0
> /dev/tty0
Environment="TERM=linux"
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

- **Explanation of systemd Configuration:**

- **[Unit] section:**

- **Description:** A human-readable description of the service.
- **After=multi-user.target:** Ensures this service starts *after* the system has reached the multi-user runlevel, meaning the console and other essential services are fully initialized.

- **[Service] section:**

- **Type=oneshot:** Indicates that the service runs a command once and then exits.
- **ExecStart=/usr/bin/setterm ... < /dev/tty0 > /dev/tty0:** This is the core command. The redirection (`< /dev/tty0 > /dev/tty0`) is crucial. It forces `setterm` to operate directly on `/dev/tty0` (the primary virtual console) rather than the systemd service's default environment, which lacks a direct terminal connection.
- **Environment="TERM=linux":** This line explicitly sets the `TERM` environment variable to `linux`, which is the standard terminal type for a physical Linux console. `setterm` requires this variable to be defined to correctly interact with the console.

- `StandardOutput=journal` and `StandardError=journal`: These directives ensure any output or errors generated by the `setterm` command are logged to the systemd journal, which is invaluable for debugging (e.g., seeing the "Inappropriate ioctl" message).
- **[Install] section:**
 - `WantedBy=multi-user.target`: Defines that this service should be enabled to start when the system enters the `multi-user.target` state (the normal server boot process).

Verification and Management

After implementing the solution, the following steps confirm its success and allow for future management:

1. Reload systemd Configuration:

```
sudo systemctl daemon-reload
```

(Required after any changes to a `.service` file).

2. Enable and Start the Service:

```
sudo systemctl enable console-power-save.service
sudo systemctl start console-power-save.service
```

(Enables it for boot and starts it immediately).

3. Reboot the Server:

```
sudo reboot
```

(To test persistence on boot).

4. Verify Status:

```
systemctl status console-power-save.service
journalctl -xeu console-power-save.service
```

(Look for `Active: inactive (dead)` and `code=exited, status=0/SUCCESS` in `systemctl status`. In `journalctl`, you might still see the "Inappropriate ioctl" warning, but the key is that the service finishes successfully and the monitor powers off).

5. Observe Physical Monitor: Confirm that after 5 minutes of inactivity, the monitor powers off, and pressing a key on the console keyboard reactivates it.

Conclusion

The `imac-server`'s console display is now configured to automatically power down after 5 minutes of inactivity using a `systemd` service that executes `set term` with appropriate console redirection and environment variables. The server remains fully operational, and the monitor wakes upon user interaction. The "Inappropriate ioctl for device" error can be safely ignored as the desired functionality is achieved.